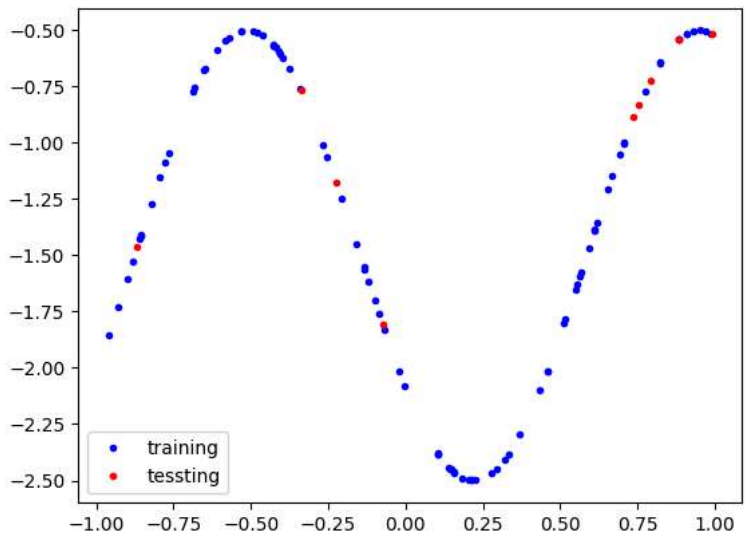
Problem 3

Prepare a simulated dataset

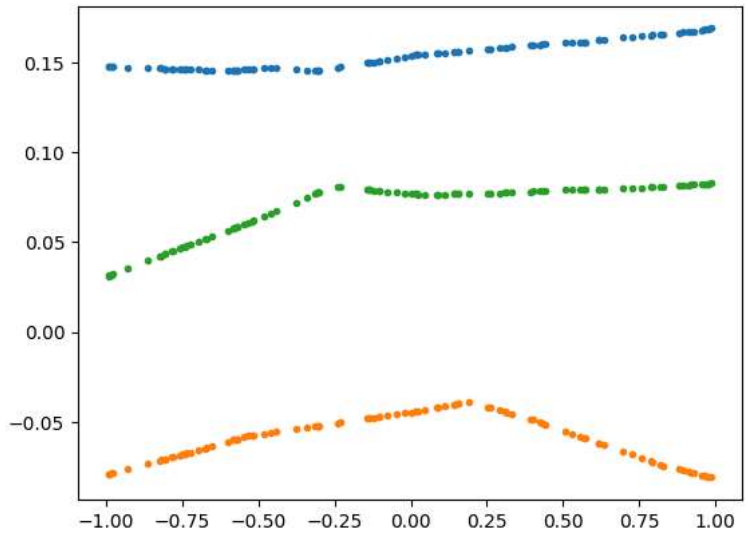
```
#@title Prepare a simulated dataset

import numpy as np
import torch
import matplotlib.pyplot as plt
# %matplotlib inline

# number of data points,
num_data = 100
train_data_ratio = 0.9
train_num = int(num_data * train_data_ratio)

# generate data
x_np = np.random.uniform(-1, 1, size=(num_data, 1))  # 1-dim data

# ground truth curve
t = 2.2
w = 4.3
b = -1.5

# use torch for target to match later ops
x_all = torch.from_numpy(x_np).float()
y_all = torch.cos(w * x_all + t) + b

# data split
train_x = x_all[:train_num, :]
train_y = y_all[:train_num, :]
test_x  = x_all[train_num:, :]
test_y  = y_all[train_num:, :]

data = dict([('train_x', train_x), ('train_y', train_y),
             ('test_x', test_x),   ('test_y', test_y)])

plt.plot(data['train_x'].numpy(), data['train_y'].numpy(), '.b', label='training')
plt.plot(data['test_x'].numpy(),  data['test_y'].numpy(),  '.r', label='tessting')
plt.legend()
plt.show()
```



Hand build a simple MLP

```
#@title Hand build a simple MLP

def relu(x):
    ''' Activation Function '''
    return x * (x > 0)
```

```
def mlp(params, x):
    '''
    params includes a list of two elements
    indicating the parameter in two layers.
    Each layer's params also have two elements
    including the weight and bias.
    '''
    w1 = params[0][0]  # weight of  layer 1, shape (hidden, in)
    w2 = params[1][0]  # weight of  layer 2, shape (out, hidden)
    b1 = params[0][1]  # bias of  layer 1, shape (hidden,)
    b2 = params[1][1]  # bias of  layer 2, shape (out,)

    h = x @ w1.t() + b1
    h = relu(h)
    y = h @ w2.t() + b2
    return y

def initialize_params(sizes, scale=1e-1, seed=1):
    ''' Initialize the weights of all layers of a linear layer network '''
    '''sizes=[input_dim, hidden_dim, output_dim]'''
    g = torch.Generator(device='cpu')
    g.manual_seed(seed)

    def initialize_layer(m, n, g, scale):
        w = scale * torch.randn(n, m, generator=g)
        b = scale * torch.randn(n,     generator=g)
        # requires_grad will be set during training
        return (w, b)

    return [initialize_layer(m, n, g, scale) for m, n in zip(sizes[:-1], sizes[1:])]

# input_dim, hidden_dim, output_dim
layer_sizes = [1, 50, 1]
# Return a list of tuples of layer weights
params_rnd = initialize_params(layer_sizes, seed=1, scale=1e-1)

# plot some neural networks with random weights.
x_plot = torch.from_numpy(np.random.uniform(-1, 1, size=(100, 1))).float()  # 1-dim data
ypred = mlp(params_rnd, x_plot).detach().numpy()
plt.plot(x_plot.numpy(), ypred, '.')
ypred = mlp(initialize_params(layer_sizes, seed=10), x_plot).detach().numpy()
plt.plot(x_plot.numpy(), ypred, '.')
ypred = mlp(initialize_params(layer_sizes, seed=100), x_plot).detach().numpy()
plt.plot(x_plot.numpy(), ypred, '.')
plt.show()
```



Gradient descent

```
#@title Gradient descent

def mse_loss(params, x, y):
```

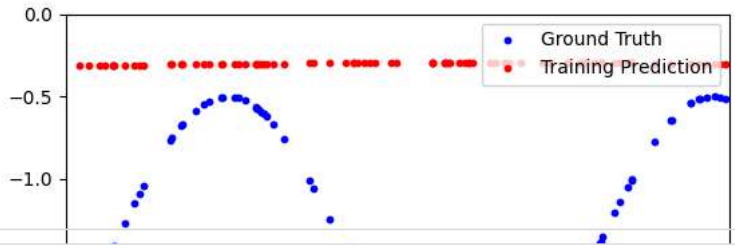
```
    ''' Define the loss function with MSE loss '''
    pred = mlp(params, x)
    return torch.mean((pred - y) ** 2)

#  One step of gradient descent
def gd_step(params, loss_fun, x, y, step_size = 1e-1):
    ''' Calculate gradient and update the parameters '''
    # enable grads
    params_rg = []
    for (w, b) in params:
        w = w.detach().clone().requires_grad_(True)
        b = b.detach().clone().requires_grad_(True)
        params_rg.append((w, b))

    loss = loss_fun(params_rg, x, y)
    loss.backward()

    params_new = []
    for (w, b) in params_rg:
        w_new = (w - step_size * w.grad).detach()
        b_new = (b - step_size * b.grad).detach()
        params_new.append((w_new, b_new))

    return params_new, loss.detach()

# Gradient descent main algorithm
def gd_train(params_initial, loss_fun, model, data, step_size=1e-1, steps=200):
    # initialization
    params = [(w.detach(), b.detach()) for (w, b) in params_initial]
    Losses_train = []
    Losses_test = []
    Pred = []

    # main loop
    for i in range(steps):
        params, loss_train = gd_step(params, loss_fun, data['train_x'], data['train_y'], step_size=step_size)
        with torch.no_grad():
            loss_test = loss_fun(params, data['test_x'], data['test_y'])

        Losses_train.append(loss_train.item())
        Losses_test.append(loss_test.item())
        with torch.no_grad():
            Pred.append(model(params, data['train_x']).detach().clone())
        if i % 100 == 0:
            print('Step {} | Training Loss:{:.2f} | Testing Loss:{:.2f}'.format(i, Losses_train[-1], Losses_test[-1]))

    trajectory = dict([('training_loss', Losses_train),
                       ('testing_loss',  Losses_test),
                       ('train_y_pred',  Pred)])
    return params, trajectory
```

Main training loop

```
#@title Main training loop

params_initial = initialize_params(layer_sizes, seed=1, scale=1e-1)
loss_fun = mse_loss
model = mlp
params, trajectory = gd_train(params_initial, loss_fun, model, data, step_size=1e-1, steps=1000)

plt.plot(trajectory['training_loss'], '.', label='training loss')
plt.plot(trajectory['testing_loss'],  '.', label='testing loss')
plt.legend()
plt.show()
```

```
Step 0 | Training Loss:2.83 | Testing Loss:0.55
Step 100 | Training Loss:0.39 | Testing Loss:0.33
Step 200 | Training Loss:0.19 | Testing Loss:0.13
Step 300 | Training Loss:0.14 | Testing Loss:0.10
Step 400 | Training Loss:0.10 | Testing Loss:0.07
Step 500 | Training Loss:0.06 | Testing Loss:0.05
Step 600 | Training Loss:0.03 | Testing Loss:0.03
Step 700 | Training Loss:0.02 | Testing Loss:0.02
Step 800 | Training Loss:0.01 | Testing Loss:0.02
Step 900 | Training Loss:0.01 | Testing Loss:0.02
```


```
# Visualize the fitting process
from matplotlib.animation import FuncAnimation
from IPython.display import HTML
import numpy as np
import matplotlib.pyplot as plt

def show_cartoon(data, trajectory):
    # cache numpy arrays once (avoid repeated .numpy() calls)
    tx = data['train_x'].detach().cpu().numpy()
    ty = data['train_y'].detach().cpu().numpy()
    preds = [p.detach().cpu().numpy() for p in trajectory['train_y_pred']]

    step = 10
    n_frames = max(1, min(len(preds) // step, 100))
    fig, ax = plt.subplots()
    sc_truth = ax.scatter(tx, ty, color='b', s=10, label='Ground Truth')
    sc_pred = ax.scatter(tx, preds[0], color='r', s=10, label='Training Prediction')

    ax.set_xlim(-1, 1)
    ax.set_ylim(-3, 0)
    ax.legend(loc='upper right')

    def animate(frame_num):
        idx = frame_num * step
        sc_pred.set_offsets(np.c_[tx, preds[idx].reshape(-1)])
        return (sc_pred,)

    anim = FuncAnimation(fig, animate, frames=n_frames, interval=60, blit=False)
    return fig, anim

fig, anim = show_cartoon(data, trajectory)

html = HTML(anim.to_jshtml())
plt.close(fig)
html
```




```
import math, torch, numpy as np, matplotlib.pyplot as plt
torch.set_default_dtype(torch.float32)

def relu(x):
    return torch.clamp(x, min=0.0)

def sigmoid(x):
    return 1.0 / (1.0 + torch.exp(-x))

def mlp(params, x, activation=relu):
    """Arbitrary-depth forward. params: list of (W, b) pairs."""
    h = x
    for i, (W, b) in enumerate(params):
        h = h @ W.t() + b
        if i < len(params) - 1:
            h = activation(h)
    return h

def initialize_params(layer_sizes, seed=0, init="normal", sigma=1e-1):
    """Init: 'normal' (σ), 'xavier', 'he'"""
    g = torch.Generator().manual_seed(seed)
    params = []
    for fan_in, fan_out in zip(layer_sizes[:-1], layer_sizes[1:]):
        if init == "normal":
            std = sigma
        elif init == "xavier":
            std = (2.0 / (fan_in + fan_out)) ** 0.5
        elif init == "he":
            std = (2.0 / fan_in) ** 0.5
        else:
            raise ValueError("init must be 'normal'|'xavier'|'he'")
        W = torch.randn(fan_out, fan_in, generator=g) * std
        b = torch.zeros(fan_out)
        params.append([W.requires_grad_(True), b.requires_grad_(True)])
    return params

def mse_loss(params, x, y, activation=relu):
    return torch.mean((mlp(params, x, activation=activation) - y)**2)

def huber_loss(params, x, y, activation=relu, delta=1.0):
    err = mlp(params, x, activation=activation) - y
    abs_err = torch.abs(err)
    quadratic = torch.minimum(abs_err, torch.tensor(delta))**2
    linear = (abs_err - delta).clamp(min=0.0) * (2.0 * delta)
    return 0.5 * torch.mean(quadratic + linear)

def zero_grads(params):
    for W, b in params:
        if W.grad is not None: W.grad.zero_()
        if b.grad is not None: b.grad.zero_()

def sgd_step(params, lr):
    with torch.no_grad():
        for W, b in params:
            W -= lr * W.grad
            b -= lr * b.grad

@torch.no_grad()
def evaluate(params, x, y, activation=relu, loss_fn=mse_loss):
    return float(loss_fn(params, x, y, activation=activation))

def train(params, train_x, train_y, test_x=None, test_y=None, *,
          activation=relu, loss_fn=mse_loss, lr=1e-2, iters=2000, log_every=500):
    traj = {"train_loss": [], "test_loss": []}
```

```
    for t in range(1, iters+1):
        loss = loss_fn(params, train_x, train_y, activation=activation)
        zero_grads(params); loss.backward(); sgd_step(params, lr)
        if (t % log_every == 0) or (t == iters):
            traj["train_loss"].append(float(loss))
            if test_x is not None:
                traj["test_loss"].append(evaluate(params, test_x, test_y, activation, loss_fn))
    return params, traj

def plot_fit(params, activation, train_x, train_y, test_x, test_y, title):
    with torch.no_grad():
        xx = torch.linspace(train_x.min(), train_x.max(), 400).unsqueeze(1)
        yy = mlp(params, xx, activation=activation)
    plt.figure()
    plt.scatter(train_x.numpy(), train_y.numpy(), s=10, alpha=0.6, label="train")
    plt.scatter(test_x.numpy(), test_y.numpy(), s=10, alpha=0.6, label="test")
    plt.plot(xx.numpy(), yy.numpy(), label="prediction")
    plt.title(title); plt.legend(); plt.show()
```

Q1) Arbitrary Layers + Architecture Sweep

```
archs = [
    [train_x.shape[1], 16, 1],
    [train_x.shape[1], 32, 1],
    [train_x.shape[1], 64, 32, 1],
    [train_x.shape[1], 128, 64, 32, 1],
]
print("Architecture sweep (ReLU + He):")
for sizes in archs:
    p = initialize_params(sizes, seed=10, init="he")
    _, tr = train(p, train_x, train_y, test_x, test_y,
                  activation=relu, loss_fn=mse_loss, lr=1e-2, iters=1500, log_every=1500)
    print(f"{sizes} -> train={tr['train_loss'][-1]:.6f} | test={tr['test_loss'][-1]:.6f}")
```

```
Architecture sweep (ReLU + He):
[1, 16, 1] -> train=0.145390 | test=0.090834
[1, 32, 1] -> train=0.033336 | test=0.026995
[1, 64, 32, 1] -> train=0.013389 | test=0.017934
[1, 128, 64, 32, 1] -> train=0.005790 | test=0.011725
```

Q2) Initialization: σ Sweep and Xavier/He

```
sigmas = (1e-5, 1e-3, 1e-1, 1.0, 10.0)
print("Sigma sweep (Normal init) for [in,32,1]:")
for s in sigmas:
    p = initialize_params([train_x.shape[1], 32, 1], seed=0, init="normal", sigma=s)
    _, tr = train(p, train_x, train_y, test_x, test_y,
                  activation=relu, loss_fn=mse_loss, lr=1e-2, iters=1200, log_every=1200)
    print(f"sigma={s:<6g} -> train={tr['train_loss'][-1]:.6f} | test={tr['test_loss'][-1]:.6f}")

sizes = [train_x.shape[1], 64, 64, 1]
p_xav = initialize_params(sizes, seed=1, init="xavier")
_, tr_xav = train(p_xav, train_x, train_y, test_x, test_y, activation=relu, loss_fn=mse_loss, lr=1e-2, iters=2000, log_every=2000)

p_he  = initialize_params(sizes, seed=1, init="he")
_, tr_he  = train(p_he,  train_x, train_y, test_x, test_y, activation=relu, loss_fn=mse_loss, lr=1e-2, iters=2000, log_every=2000)

print(f"Xavier vs He (ReLU): Xavier test={tr_xav['test_loss'][-1]:.6f} | He test={tr_he['test_loss'][-1]:.6f}")
```

```
Sigma sweep (Normal init) for [in,32,1]:
sigma=1e-05  -> train=0.496594 | test=0.369501
sigma=0.001  -> train=0.496502 | test=0.369643
sigma=0.1    -> train=0.290581 | test=0.222587
sigma=1      -> train=0.020584 | test=0.020165
sigma=10     -> train=0.496594 | test=0.369618
Xavier vs He (ReLU): Xavier test=0.014272 | He test=0.017267
```

Q3) Swap ReLU for Sigmoid and Tune

```
sizes = [train_x.shape[1], 64, 64, 1]

# ReLU baseline
p_relu = initialize_params(sizes, seed=2, init="he")
_, relu_tr = train(p_relu, train_x, train_y, test_x, test_y,
                   activation=relu, loss_fn=mse_loss, lr=1e-2, iters=2000, log_every=2000)

# Sigmoid
p_sig = initialize_params(sizes, seed=2, init="xavier")
_, sig_tr = train(p_sig, train_x, train_y, test_x, test_y,
                  activation=sigmoid, loss_fn=mse_loss, lr=5e-3, iters=2500, log_every=2500)

print(f"ReLU test={relu_tr['test_loss'][-1]:.6f}  |  Sigmoid test={sig_tr['test_loss'][-1]:.6f}")
```

```
ReLU test=0.015683  |  Sigmoid test=0.384638
```

Q4) Huber Loss vs MSE with Outliers

```
def with_outliers(y, k=5, jump=10.0):
    y2 = y.clone()
    idx = torch.arange(min(k, y2.shape[0]))
    y2[idx] = y2[idx] + jump
    return y2

train_y_out = with_outliers(train_y, k=5, jump=10.0)
sizes = [train_x.shape[1], 64, 64, 1]

p_mse = initialize_params(sizes, seed=3, init="he")
_, mse_tr = train(p_mse, train_x, train_y_out, test_x, test_y,
                  activation=relu, loss_fn=mse_loss, lr=1e-2, iters=2000, log_every=2000)

p_huber = initialize_params(sizes, seed=3, init="he")
_, huber_tr = train(p_huber, train_x, train_y_out, test_x, test_y,
                    activation=relu, loss_fn=lambda P,X,Y,activation=relu: huber_loss(P,X,Y,activation,delta=1.0),
                    lr=1e-2, iters=2000, log_every=2000)

print(f"Outliers → MSE test={mse_tr['test_loss'][-1]:.6f}  |  Huber test={huber_tr['test_loss'][-1]:.6f}")
```

```
Outliers → MSE test=0.071892  |  Huber test=0.016966
```